

Exercise 7: Packages

Scenario 1: Group all customer-related procedures and functions into a package.

Question: Create a package **CustomerManagement** with procedures for adding a new customer, updating customer details, and a function to get customer balance.

Package Specification

```
CREATE OR REPLACE PACKAGE CustomerManagement AS
```

```
    PROCEDURE AddCustomer(
```

```
        p_customerID NUMBER,
```

```
        p_name VARCHAR2,
```

```
        p_dob DATE,
```

```
        p_balance NUMBER
```

```
    );
```

```
    PROCEDURE UpdateCustomer(
```

```
        p_customerID NUMBER,
```

```
        p_balance NUMBER
```

```
    );
```

```
    FUNCTION GetCustomerBalance(
```

```
        p_customerID NUMBER
```

```
    ) RETURN NUMBER;
```

```
END CustomerManagement;
```

Package Body

```
CREATE OR REPLACE PACKAGE BODY CustomerManagement AS
```

```
    PROCEDURE AddCustomer(
```

```
p_customerID NUMBER,  
  
p_name VARCHAR2,  
  
p_dob DATE,  
  
p_balance NUMBER  
  
)  
  
IS  
  
BEGIN  
  
    INSERT INTO Customers  
  
VALUES (  
  
    p_customerID,  
  
    p_name,  
  
    p_dob,  
  
    p_balance,  
  
    SYSDATE  
  
    );  
  
END;  
  
PROCEDURE UpdateCustomer(  
  
    p_customerID NUMBER,  
  
    p_balance NUMBER  
  
)  
  
IS  
  
BEGIN  
  
    UPDATE Customers  
  
    SET Balance = p_balance  
  
    WHERE CustomerID = p_customerID;  
  
END;
```

```
FUNCTION GetCustomerBalance(  
    p_customerID NUMBER  
)  
RETURN NUMBER  
IS  
    v_balance NUMBER;  
BEGIN  
    SELECT Balance  
    INTO v_balance  
    FROM Customers  
    WHERE CustomerID = p_customerID;  
    RETURN v_balance;  
END;  
END CustomerManagement;
```

Scenario 2: Employee Management Package

Scenario 2: Create a package to manage employee data.

Question: Write a package **EmployeeManagement** with procedures to hire new employees, update employee details, and a function to calculate annual salary.

Package Specification

```
CREATE OR REPLACE PACKAGE EmployeeManagement AS  
    PROCEDURE HireEmployee(  
        p_employeeID NUMBER,  
        p_name VARCHAR2,
```

```
        p_position VARCHAR2,

        p_salary NUMBER,

        p_department VARCHAR2,

        p_hireDate DATE

    );

PROCEDURE UpdateEmployee(

    p_employeeID NUMBER,

    p_salary NUMBER

);

FUNCTION CalculateAnnualSalary(

    p_employeeID NUMBER

) RETURN NUMBER;

END EmployeeManagement;
```

Package Body

```
CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS

    PROCEDURE HireEmployee(

        p_employeeID NUMBER,

        p_name VARCHAR2,

        p_position VARCHAR2,

        p_salary NUMBER,

        p_department VARCHAR2,

        p_hireDate DATE

    )

    IS
```

BEGIN

INSERT INTO Employees

VALUES (

p_employeeID,

p_name,

p_position,

p_salary,

p_department,

p_hireDate

);

END;

PROCEDURE UpdateEmployee(

p_employeeID NUMBER,

p_salary NUMBER

)

IS

BEGIN

UPDATE Employees

SET Salary = p_salary

WHERE EmployeeID = p_employeeID;

END;

FUNCTION CalculateAnnualSalary(

p_employeeID NUMBER

)

RETURN NUMBER

IS

```
v_salary NUMBER;  
  
BEGIN  
  
    SELECT Salary  
  
    INTO v_salary  
  
    FROM Employees  
  
    WHERE EmployeeID = p_employeeID  
  
    RETURN v_salary * 12;  
  
END;  
  
END EmployeeManagement;
```

Scenario 3: Account Operations Package

Scenario 3: Group all account-related operations into a package.

Question: Create a package **AccountOperations** with procedures for opening a new account, closing an account, and a function to get the total balance of a customer across all accounts.

Package Specification

```
CREATE OR REPLACE PACKAGE AccountOperations AS
```

```
    PROCEDURE OpenAccount(  
  
        p_accountID NUMBER,  
  
        p_customerID NUMBER,  
  
        p_accountType VARCHAR2,  
  
        p_balance NUMBER  
  
    );
```

```
    PROCEDURE CloseAccount(  
  
        p_accountID NUMBER,  
  
        p_customerID NUMBER,  
  
        p_balance NUMBER  
  
    );
```

```
        p_accountID NUMBER
    );

    FUNCTION GetTotalBalance(
        p_customerID NUMBER
    ) RETURN NUMBER;

END AccountOperations;
```

Package Body

```
CREATE OR REPLACE PACKAGE BODY AccountOperations AS
```

```
    PROCEDURE OpenAccount(
        p_accountID NUMBER,
        p_customerID NUMBER,
        p_accountType VARCHAR2,
        p_balance NUMBER
    )
```

```
    IS
```

```
    BEGIN
```

```
        INSERT INTO Accounts
```

```
    VALUES (
```

```
        p_accountID,
```

```
        p_customerID,
```

```
        p_accountType,
```

```
        p_balance,  
        SYSDATE  
    );  
END;
```

```
PROCEDURE CloseAccount(  
    p_accountID NUMBER  
)  
IS  
BEGIN  
    DELETE FROM Accounts  
    WHERE AccountID = p_accountID;  
END;
```

```
FUNCTION GetTotalBalance(  
    p_customerID NUMBER  
)  
RETURN NUMBER  
IS  
    v_total NUMBER;  
BEGIN  
    SELECT SUM(Balance)  
    INTO v_total  
    FROM Accounts  
    WHERE CustomerID = p_customerID;
```



```
RETURN NVL(v_total, 0);
```

```
END;
```

```
END AccountOperations;
```